Harness Engineering 实操手册

如何组织AI与人类Agent完成可靠的软件交付

Agents特区社区集体智慧 | 执笔人：瑞典马工

问题

社区内普遍报告：使用AI写代码后，PR合并量上升、review时间变长、bug率走高，综合生产力提升有限。

AI加速的是软件开发生命周期里最便宜的环节：写代码。贵的环节，设计、验证、调试、部署、维护，速度没变。

这篇文章试图回答一个核心问题：**怎么组织AI和人类一起做可靠的软件交付？**

答案围绕四个支柱展开，按依赖关系排列：Spec（做什么）→ Verification（对不对）→ Orchestration（怎么组织）→ Evolution（怎么变好）。

愿景：业务影响

这套框架从速度、质量与成本三个维度改变交付经济学：

- 更快的收入周期：合格需求数小时/数天内变成可运行演示；更多订单当季签约。
- 更高赢率与更大单：范围可定价、可交付；SLA可验证，买方信任更强。
- 规模化、可盈利的定制：按租户以规格/规则定义变体（零分叉），升级无痛，定制有利润。
- 可预测、低风险交付：阶段闸门+自动化验证压降返工与事故；承诺可兑现。
- 更低单位产出成本、可扩容产能：小团队编排大量Agent；在不降质前提下提升吞吐。

执行模型：敏捷需求，瀑布任务

需求阶段用敏捷方式迭代探索。每个任务一旦进入执行，严格按顺序推进（瀑布）：

Spec → ⍰⍰ → ⍰⍰ → ⍰收 → 部署

Spec是分界线。Spec之前：自由迭代。Spec之后：严格执行。AI agent在紧约束下表现最佳；模糊的流程导致上下文污染和偷工减料。

阶段闸门（Stage Gates）

- 进入编码前置条件（Gate 1）：存在机器可读的验收标准种子（至少1个Trophy测试雏形 + 2 - 3条核心AC）。
- 回退路径：若测试揭示Spec歧义/缺口，流程回到Spec阶段；暂停编码，待Spec更新并经评审确认后再前进。
- 阶段内小循环：允许在同一阶段内的红?绿?重构循环，但禁止跨闸门跳跃（例如在无AC时直接合并代码）。
- Spike路径（一次性试探）：允许快速原型但产物默认可丢弃；复用前必须补齐Spec与测试并重新过闸门。

最小可运行基线（MOB）

- 1份机器可读Spec（任务ID、上下文、输入、约束、验收标准、非目标）。
- 1个Trophy级端到端测试（用户可感知能力）。
- CI在PR上执行测试；至少一种对抗/突变检查以防“表面通过”。
- 变更日志：PR需链接所触达的Spec/Skill条目；测试/Spec不同步时禁止合并。

支柱一：Specification（规格）

问题：做什么？

实践	描述
机器可读的PRD	Spec是AI的操作手册，不是给人看的沟通文档
验收标准即评估函数	可测试的完成定义，直接成为测试用例的种子
API契约即信任边界	接口规格 > 实现细节（Design by Contract）
文档测试 [1]	用另一个AI模型对spec进行"思想实验"——不写代码，纯粹基于文档"运行"系统以发现逻辑漏洞。成本：仅消耗token

核心洞察： 瓶颈已从执行速度转移到规格清晰度。

可借鉴原则： 任务说明书、契约式设计。

机器可读PRD最小结构（示例）

```
task_id: FEAT-123
context: ["/docs/arch.md#payments", "/tickets/9876"]
inputs:
  - name: amount
    type: decimal
invariants:
  - amount > 0
acceptance_criteria:
  - "支付成功后在⍰⍰⍰出⍰一条金⍰=amount、状⍰=success的⍰⍰"
  - "失⍰⍰返回可本地化的⍰⍰⍰，未写入⍰⍰"
non_goals:
  - "不⍰理分期与退款"
```

文档测试的护栏

- 目的：在编码前暴露逻辑矛盾与缺口；不替代执行级验证。
- 限制：可能产生幻觉；必须与人类编写AC对齐；任何发现需回写Spec。

常见失效模式与控制

- 陈旧：Spec与实现漂移 → 设每周审查；PR必须链接触达的Spec段落。
- 歧义：自然语言多义 → 使用示例/反例与边界条件；关键字段结构化。
- 冲突：多源文档优先级不明 → 引入信任环与单一信源（Ring 0）。
- 表面积膨胀：无关需求泄漏 → 维护非目标（Non-goals），限制Scope。

支柱二：Verification（验证）

问题：对不对？

质量的两个维度：

维度	问题	可自动化？	AI能力
Correctness（正确性）	代码能跑吗？	完全可以	强
Fitness（适用性）	产品对吗？	部分可以	弱

大多数团队只验证Correctness，在Fitness上积累**验证债务**——"测试全绿，但你精确地构建了spec描述的东西，却不是客户真正想要的。"

实践	描述
TDD（测试先行）	编码成本→0，测试的相对价值→∞
Trophy测试模型	集成测试是AI最难作弊的层级（Goodhart定律）
对抗性验证	政委角色：独立agent对照spec检查其他agent的产出
跨模型验证	用不同模型验证，避免同模型共谋
回归测试	没有回归测试的AI迭代 = 混乱

Ground Truth悖论： AI写代码 + AI写测试 = 共谋。社区实践观察到模型可能修改/删除测试以“看起来通过”。解法：测试必须锚定在人类编写的验收标准（支柱一）上，而非AI生成的代码上。

做法： 闭环验证（计划-执行-检查-行动）能显著降低缺陷；验证的多样性应匹配潜在错误的多样性。

Trophy测试模型（定义）

- 面向用户可感知能力；端到端；任一关键子路径失败即整体失败；难以通过“走捷径”来作假。

交叉验证与合并规则

- 跨模型验证：使用不同家族/提供商模型相互检查，降低相关错误风险；记录差异与裁决理由。
- 合并门槛：Correctness全绿仍不足以合并；至少需要1个Fitness代理信号（端到端指标、真实数据回放、利益相关者验收）为绿，或经明确豁免并记录理由。

失效模式与对策

- 测试过拟合实现细节 → 以行为为断言，避免断言内部状态；加入突变测试/对抗样例。
- 共谋：AI写代码+AI写测试 → 验收标准锚定人类AC；交叉模型与对抗提示。
- 回归缺失：修复无回归 → 任何bug修复必须伴随回归用例。

支柱三：Orchestration（编排）

问题：怎么组织？

实践	描述
Skill即岗位说明书	每个Skill：明确的输入、输出、职责边界
Subagent委派	明确的任务说明 + 验收标准 + 完成验证
平台工程前置	在写业务代码之前先搭建CI/CD、测试框架、监控
3-5人小组 [2]	小型人类团队编排AI agent
小时级团队调配	以小时为周期迭代团队组成，而非以月为周期
HR角色	专门负责组建和调整agent团队的职能

关键约束： 管理幅度受限于验证自动化与CI稳定度。社区经验：通过自动化验证（支柱二）扩大管理幅度，而非增加人力。

Conway定律依然适用。 系统架构映射团队结构。先重组团队，否则工作流将被旧组织架构所约束。

Skill卡片最小结构（示例）

```
skill: Coder
inputs: [spec.section, test.failures]
outputs: [pr.diff, test.results]
definition_of_done: ["⍰⍰全⍰", "PR描述映射到Spec条目"]
trust_boundaries: ["不更改数据⍰schema", "不修改外部API契⍰"]
failure_retry: {retries: 2, strategy: "⍰小⍰更面+⍰求更多上下文"}
escalation: "⍰⍰失⍰⍰交Reviewer并回Spec澄清"
budgets: {latency_s: 300, cost_usd: 1.5}
```

管理幅度的前置条件

- 仅在以下成立时扩大span：AC覆盖率≥阈值、CI稳定（低波动/低假阳）、领域稳定（需求变更速率受控）、任务种类有界。
- 否则以小步快跑为主，先投资于验证自动化与平台工程。

运行约束

- 成本/时延预算；模型/工具多样性策略；数据/权限治理（最小必要上下文）。

支柱四：Evolution（演进）

问题：怎么变好？

实践	描述
Skill标准化	个人隐性知识 → 共享的显性Skill
迭代式团队组建	从简单开始，积累经验，调整角色
双环学习	不只是修bug——还要质疑产生bug的架构与流程
行动后回顾	每次agent工作session后的结构化复盘
知识持久化	Skill弥补SECI断裂——agent跨session丢失记忆

跨Session断裂： Agent无法在会话间保留隐性知识。Skill是工程化的补救方案——组织知识的持久载体。

没有演进，系统退化。 AI会复制代码库中的模式，包括坏模式；合并后的代码成为后续生成的参考集，形成自强化的质量下降循环。如果不持续改进Spec、测试与Skill，代码库会自己滑向平庸。

行动后回顾（10分钟模板）

- 发生了什么？与Spec/测试期望的差异？
- 哪个假设错了？证据是什么？
- 下次要改变的一个行为是什么？
- 哪个Skill/Spec/测试需要更新？由谁在何时更新？
- 需要新增哪条回归或对抗用例？

SECI断裂的工程化弥补

- 将反复出现的隐性做法固化为Skill卡片；每周评审一次；当PR引入行为变化时要求链接到对应的Skill更新。

贯穿性关切

上下文污染——要防止的病态

更多上下文 ≠ 更好的结果。社区实验表明：冗长或噪声上下文会降低成功率并提高推理成本，即使远未达到token上限。

六种污染类型：

类型	描述	对抗机制
过时污染	过时的文档、注释、架构图	熵管理，周期性清理
冲突污染	多信息源优先级不明	信任环（Ring 0-3），单一信源
推理污染	跨文档推理错误	执行时展开为扁平文档
压缩污染	上下文压缩过程中的选择性信息丢失	基于优先级的文档分层（P0-P3）
共谋污染	AI生成的测试验证AI生成的代码	人类编写验收标准，突变测试
遗留污染	废弃模式、死代码被AI吸收	回归测试，积极删除

类比： 分布式系统中的拜占庭故障。AI不是不工作——它自信地产出错误结果。

检测信号与控制（每类各举一例）：

- 过时：Spec与代码差异/周上升 → 每周Spec审计与废弃文档墓碑（tombstone）。
- 冲突：同一需求多处不一致 → 引入“单一信源”并在PR中检查引用。
- 推理：跨文档推导错误 → 执行时扁平化上下文，减少跨层依赖。
- 压缩：长上下文裁剪丢关键信号 → P0/P1优先级分层与强制保留集。
- 共谋：测试与实现同源 → 交叉模型+对抗/突变测试作为阻断器。
- 遗留：旧API/死代码被引用 → 设定节奏化的死代码清扫与API下线流程。

反馈循环——要建设的机制

循环	时间尺度	功能
内环	秒-分钟	Agent自主循环：红-绿-重构
中环	小时-天	人类review，方向调整
外环	周-月	战略演进，Skill迭代

支柱间的反馈：Spec → 驱动测试设计 → 测试结果驱动流水线决策 → 观察到的模式驱动Skill改进 → 演进更新Spec。

Vibe coding = 开环控制。 本框架 = 闭环控制。

闭环/开环的可判定标准与停止条件：

- 开环判据：代码变更未伴随Spec/测试变更记录；或仅凭主观感受合并。
- 内环停止：Trophy + AC通过，回归不回退；中环停止：Reviewer可无新增假设地将产出映射回Spec。

社区争议

本框架不是共识。以下是社区中有代表性的反对意见，每一条都指向框架的真实边界。

1. "抛弃旧经验" vs "管理学原理迁移"

一种观点：管理学原理依然适用于AI编排。另一种观点（社区内）：必须抛弃既有软件工程经验，承认AI是“新物种”。这不是程度之争，而是检验旧框架能否承载新实践。回应：原理与实践是两个层次；任务分解与反馈控制等原理跨领域有效，而具体实践（如站会、看板）需要重设计。

验证计划：在两个团队上做A/B：A 采用“原地分层执行”，B 采用“设计时分层、执行时扁平化”，对比成功率与推理/成本。

2. "分层是给人用的，不是给LLM用的"

胥克谦在社区中提出：文档分层帮助人类组织思维，但LLM执行时应展开为扁平的单一文档，否则跨层推理引入错误。这直接挑战了Spec分层实践——如果分层在执行阶段反而有害，它到底是设计工具还是执行工具？框架已将此纳入"推理污染"类型，但"设计时分层、执行时展开"的实操成本尚未得到验证。

验证计划：以“先评估后试点”与“先试点后评估”两种路径在相近场景对比3个月的缺陷逃逸率与重工率。

3. Benchmark是否是前置条件

框架隐含"评估先行"——先建立验证体系，再放手让AI干活。大史在社区中质疑：团队内部落地Agent赋能，是否必须先有靠谱的Benchmark？王津银直言"不一定"——在传统组织中，IT部门很难对强势的业务部门做Benchmark，关键是先用3个月自证能力，获得话语权后再谈评估。这揭示了框架的学术偏见：理论上评估应该先行，实践中权力结构决定落地顺序。

4. 感知与现实的脱节

有从业者报告：主观感觉更快，客观测量却更慢。这一现象挑战了“AI一定提升吞吐”的前提。

开放问题

1. **递归验证。** 谁来监督监督者？终止条件是什么？
2. **最优上下文量。** 太少 = 信息不足。太多 = 噪声淹没信号。最优点在哪？
3. **人类能力退化。** 人类判断力在下降（理解力-17%），AI自主性在上升。Human-in-the-loop还能维持多久？
4. **框架边界。** 有没有实践落不到这四个支柱里？

试验协议（针对问题2）：

- 选定固定任务集；设定两档上下文预算（P50与P90长度）；测量成功率、回合数、token成本与缺陷逃逸；报告差异与显著性。

治理与可追溯（简）

- 版本化：Spec/Skill版本与PR绑定；合并即生成变更记录。
- 风险约束：安全关键需求走额外流程；定义成本/时延预算；工具/模型多样性策略。
- 数据治理：最小必要上下文原则；敏感数据不入公共上下文；权限与审计开启。

本文由Agents特区社区共创。 实践和争议均来自社区讨论。框架仍在演进，欢迎补充、反驳、纠正。

---

1. 文档测试概念来自胥克谦（Agents特区社区）。[?]

2. "三人行"小团队模式来自王津银（Agents特区社区企业转型实践）。[?]